

Overview of the relational operator search procedure

- Parse the query and organize it into a bunch of lists and dictionaries for later use
- Loop over the DataFed records and find where the parameters match (parameters are “Ablation_Temperature”, “Base_Pressure”, “Chamber”, etc...)
- Record the parameters and values of the matches in a dictionary (values are 700, 210, “Laser_1C”, etc.)
- construct strings setting up the relational operators working on the query and DataFed matches.
- Hide the matches that evaluate to false

Detailed description of the relational operator search procedure

- Define dictionaries mapping relational operators to the corresponding functions from the built-in “operator” library for numeric, string, and date data for later use
- Parse and organize the search query
 - Identify search quantity, relational operator, value, parenthesis, etc and put in appropriate array and then dictionary
 - ♣ Standardize conjunctions (for searching multiple terms):
 - all instances of ‘&’ become “and”
 - all instances of “|” become “or”
 - all instances of “^” become “xor”
 - ♣ Split the search query (“searchStr”) on the now standardized conjunctions into an array (“searchArray”)
 - ♣ Identify parenthesis and record their indices into arrays (“beginning_parens_indices” and “ending_parens_indices”)
 - ♣ Loop over the searchArray and separate the different parts into arrays
 - Store the conjunctions in a list (“conj_list”)
 - Standardize the relational operators to accept multiple representations (i.e. any number of equals signs for equality) and store them in a list (“relOp”)
 - Store each element of searchArray besides the conjunctions in a list (“search_array_without_conj”)
 - Identify the searched parameters and store them in a list (“parameters_list”)
 - Store the conjunctions in an array (“conj_list”)
 - ♣ Standardize the search quantities to match what they are called in DataFed and store those in a list (“parameter_array”)
 - ♣ Define the list “string_value_array” to record whether the number is actually a string based on the quantity
 - ♣ Identify the searched value and store it in an array (“number_array”).
 - ♣ Convert these lists to dictionaries, where the key is the search parameter, and the value is the searched value, relational operator, or conjunction (“number_dict”, “relOp_dict”, and “conj_dict”)
 - if one of the parameters in parameter_array is a list (for example if you just search “pulses”, the parameter will be [“Ablation_Pulses”, “Pre_Ablation_Pulses”]) , pick the shortest element (“Ablation_Pulses”) to be the dictionary key for simplicity

- if you search for the term multiple times (for example temp > 600 or temp < 800 so that the key already exists, make a list with both values so for example:
 - o number_dict = {'Ablation_Temperature': [600,800]}
 - o relOp_dict = {'Ablation_Temperature':['>','<']}
 - o conj_dict = {'Ablation_Temperature':or}
- TODO: this could probably be streamlined by creating the dictionaries from the start instead of starting with the lists and then converting them to dictionaries.
- ♣ Create a flattened parameter array ("parameter_array_flattened") so that later we can iterate over each value instead of nested lists
- Loop over the DataFed collections in a project
 - o Unhide the project
 - o For a given collection, loop over the data records
 - ♣ Unhide the data record
 - ♣ Instantiate some empty dictionaries and lists to hold the data and perform the metadata. They are all populated several steps later but are defined here so that they are emptied before starting to parse each record. They are defined below:
 - metadata: a dictionary to store the matched datafed quantities. The key is the corresponding metadata header ("Header","Target_1") and the value is another dictionary where the key is the quantity ("User_Name", "Ablation_Pulses") and the value is the value.
 - parameter_indices: a list to store the index. This occurs a couple steps later but is defined here so that it is emptied before starting to parse each record.
 - metadata_dict: a dictionary to store the datafed quantities. The key is the corresponding metadata header ("Header","Target_1") and the value is the quantity ("User_Name", "Ablation_Pulses")
 - eval_strings: a list to store input strings for the build in eval function to determine whether there is a logical match between the searched quantities and the DataFed metadata quantities. The strings are defined as "eval_str" below
- ♣ For a given record, loop over the metadata sections ("Header","Target_1", etc.)
 - Unhide the metadata headers
 - Loop over the metadata parameters ("Ablation_Temperature", etc.)
 - o Unhide the metadata section
 - o Create a list to store the metadata parameters "DataFed_parameter_list"
 - o Assume someone won't search for a section so loop over the parameters
 - Unhide the metadata parameters
 - o Add the metadata section and the list of parameters in that DataFed record ("DataFed_parameter_list") to metadata_dict as a key:value pair
 - If there is no metadata (resulting in an empty metadata_dict) for a record, hide the record
 - o TODO: this could probably happen earlier to save computation
 - Instantiate and then iteratively fill a string "eval_str" with statements for the built-in eval function to determine whether there is a logical match between the searched parameters and the DataFed metadata parameters according to the parentheses and

conjunctions of the search. This determines whether that DataFed parameter is in the query.

- Append “eval_str” to the previously defined “eval_strings”
- ♣ Loop over the sections again, this time ensuring that eval_str is True, so at least the parameters match, and evaluate the value
 - TODO: these two loops over the sections could probably be combined so that it is only necessary to loop over the sections once.
 - If the eval_str for that section is not empty and evaluates to True, define a dictionary (“concatenated_DataFed_parameters”) that is just metadata_dict[“Header”] for headers in headers in Header and metadata_dict[“Header”] + metadata_dict[“Target_{i}”] for i in the targets in the DataFed record
 - Loop over the DataFed parameters (“Chamber”, “Ablation_Temperature”, etc”)
 - If the DataFed_parameter is in parameter_array and therefore also the search query and consequently a match, loop over both the parameter_array (a nested list containing what each query parameter is called on DataFed) and the concatenated_DataFed_parameters (a list containing the parameters for that DataFed section).
 - If DataFed_parameter == concatenated_DataFed_parameter, there is a match, so loop over the DataFed_values (values are “Oxygen, Laser_1C, Value, etc.”)
 - If the values have no children, the metadata has no units
 - If DataFed_value is a float, add an entry to metadata where the key is the section and the value is another dictionary where the key is the DataFed parameter and the value is the DataFed_value converted to float
 - Elif DataFed_parameter is “Date”, add an entry to metadata where the key is the section and the value is another dictionary where the key is the DataFed parameter and the value is the DataFed_value converted to datetime format
 - Else, DataFed_value is a string, so add an entry to metadata where the key is the section and the value is another dictionary where the key is the DataFed parameter and the value is the DataFed_value converted to casefold format
 - Otherwise, DataFed_value has units, so loop over them
 - It is pointless to search for the unit itself so skip to where DataFed_value == Value
 - If DataFed_value is a float, add an entry to metadata where the key is the section and the value is another dictionary where the key is the DataFed parameter and the value is the DataFed_value converted to float
 - Otherwise, DataFed_value is a string, so add an entry to metadata where the key is the section and the value is another dictionary where the key is the DataFed

parameter and the value is the DataFed_value converted to casefold format

- Put the matched section, DataFed parameter and DataFed value into the “metadata” dictionary. The key is the section and the value is a dictionary with the key as the parameter and the value as the value
- If there is no match, hide the section
- ♣ Loop over the keys and nested keys in the metadata dictionary to iteratively create a string “metadata_eval_str” that contains the relational operators, DataFed values, and query values in the proper format for eval to determine whether the match evaluates to True or False
- ♣ Create a new dictionary “metadata_eval_dict” that connects the datafed and query values for each match. The key is the parameter and the value is a dictionary with the section as the key and the value is the metadata_eval_str
- ♣ Convert metadata_eval_dict into an OrderedDict where the keys are in numerical order instead of string order, so the keys are:
Header, Target_1, Target_2, Target_3, Target_4, Target_5, Target_6, Target_7, Target_8, Target_9, Target_10, Target_11, Target_12
instead of
Header, Target_1, Target_10, Target_11, Target_12, Target_2, Target_3, ...
for ease of iterating later
- ♣ Instantiate “Metadata_eval_str_{i}” for each target (Assuming max 12 targets for now)
- ♣ Loop over the relOp_dict keys
 - If the key is not also in the metadata_eval_dict
 - If the key is in the conj_dict keys and the corresponding conjunction is “and,” the metadata only has some of the parameters but not all of them. Since the conjunction is “and”, we can directly set the Metadata_eval_str_{i} for each target i to “False” and move on to the next key
 - Otherwise, loop over the metadata_eval_dict subkeys (these are “Header”, “Target_1”, “Target_2”, ...)
 - Populate the relevant Metadata_eval_str_{i} for each subkey (making sure each has a header) with the string ready for evaluation from metadata_eval_dict[key1][key2] (the nested value from metadata_eval_dict) connected by the conjunctions from conj_dict as appropriate.
- ♣ Loop over the “Metadata_eval_str_{i}” for i in range(1,13)
 - If the Metadata_eval_str_{i} is not empty, put the opening and parentheses in the appropriate places using beginning_parens_indices and ending_parens_indices
 - If the Metadata_eval_str_{i} evaluates to False, remove the DataFed section from the “metadata” dictionary (these are the dictionary keys)
 - If the only key is “Header” and other keys have been deleted, then hide the “Header” as well
- ♣ If the only key in metadata is “Header”, then search term is only in the Header so record the record as not hidden
- ♣ Otherwise, the metadata has many sections, so loop over the sections in the DataFed record
 - If the section is not in the metadata keys, hide the section
 - Otherwise, unhide the section

- ♣ If all of the sections are hidden, hide the record
- o If all of the records are hidden, hide the collection